

Database Management System Lab (CS 29006)

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

School of Computer Engineering



Strictly for internal circulation (within KIIT) and reference only. Not for outside circulation without permission

1 Credit

Course Committee

Lab Contents



2

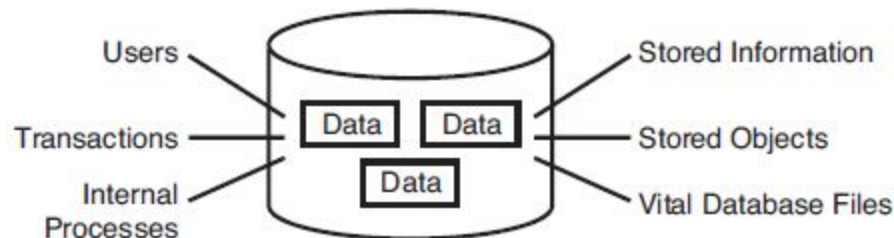
Sr #	Major and Detailed Coverage Area	Lab#
1	• An introduction to SQL • Introduction to database management systems • Overview of some basic terms and concepts	1

DBMS



3

In simple terms, a **database** is a collection of data. A phone book is a database. The data contained consists of individuals' names, addresses, and telephone numbers. Every modern-day business has data, which requires some organized method or mechanism for maintaining and retrieving the data. When the data is kept within a database, this mechanism is referred to as a **database management system** (DBMS). The database has to be maintained. As people move to different cities or states, entries might have to be added or removed from the phone book. Likewise, entries have to be modified for people changing names, addresses, telephone numbers, and so on.

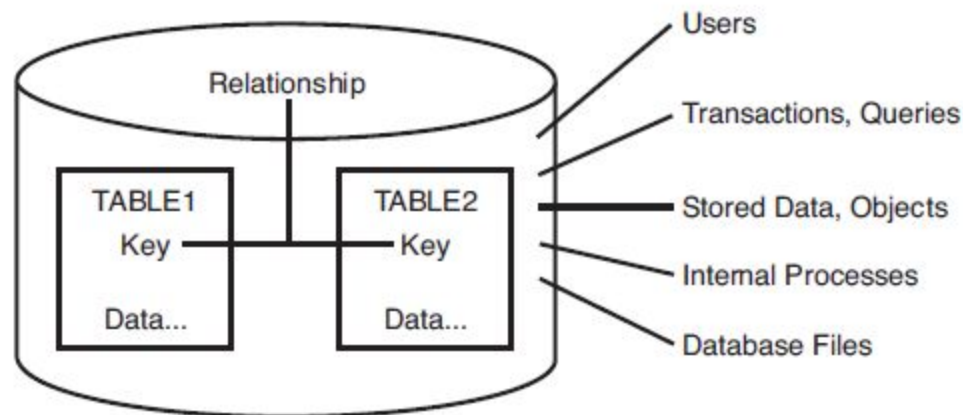


RDBMS



4

The modern wave of information management is primarily carried out through the use of a relational database management system (**RDBMS**), derived from the traditional DBMS. A relational database is a database divided into logical units called tables, where tables are related to one another within the database. A relational database allows data to be broken down into logical, smaller, manageable units, enabling easier maintenance and providing more optimal database performance according to the level of organization.



The relational algebra provides a concise, formal notation for representing queries. However, commercial database systems require a query language that is more user friendly. SQL can do much more than just query a database. It can define the structure of the data, modify data in the database and specify security constraints.

What is SQL?

Structured Query Language (SQL) is the standard language used to communicate with a relational database. The prototype was originally developed by IBM using Dr. E.F. Codd's paper ("A Relational Model of Data for Large Shared Data Banks") as a model. In 1979, not long after IBM's prototype, the first SQL product, ORACLE, was released by Relational Software, Incorporated (which was later renamed Oracle Corporation). Today it is one of the distinguished leaders in relational database technologies.

SQL Command Types



6

The basic categories of commands used in SQL to perform various functions are

- ☐ Data Definition Language (DDL)
 - ☐ Data Manipulation Language (DML)
 - ☐ Data Query Language (DQL)
- } **Will cover**

- ☐ Data Control Language (DCL)
 - ☐ Data administration commands
 - ☐ Transactional control commands
- } **Out of course and will cover if time permits**

DDL & DML



7

Data Definition Language (DDL) is the part of SQL that enables a database user to create and restructure database objects, such as the creation or the deletion of a table. Some of the most fundamental DDL commands include:

- | | | |
|---------------------------------------|---------------------------------------|--------------------------------------|
| ☐ Create Table | ☐ Create Index | ☐ Create View |
| ☐ Alter Table | ☐ Alter Index | ☐ Alter View |
| ☐ Drop Table | ☐ Drop Index | ☐ Drop View |

Data Manipulation Language (DML) is the part of SQL used to manipulate data within objects of a relational database. The three basic DML commands are:

- ☐ INSERT
- ☐ UPDATE
- ☐ DELETE

DQL and DCL



8

DQL - Though comprised of only one command, **Data Query Language** (DQL) is the most concentrated focus of SQL for modern relational database users. The base command is SELECT. This command, accompanied by many options and clauses, is used to compose queries against a relational database. A query is an inquiry to the database for information. A query is usually issued to the database through an application interface or via a command-line prompt. You can easily create queries, from simple to complex, from vague to specific.

DCL - Data control commands in SQL enable you to control access to data within the database. These **Data Control Language** (DCL) commands are normally used to create objects related to user access and also control the distribution of privileges among users. Some data control commands are as follows:

- ☐ ALTER PASSWORD
- ☐ REVOKE
- ☐ GRANT
- ☐ CREATE SYNONYM

Data administration commands

9

Data administration commands enable the user to perform audits and perform analyses on operations within the database. They can also be used to help analyze system performance. Two general data administration commands are as follows:

- ☐ START AUDIT
- ☐ STOP AUDIT

Do not get data administration confused with database administration. Database administration is the overall administration of a database, which envelops the use of all levels of commands. Data administration is much more specific to each SQL implementation than are those core commands of the SQL language.

Transactional Control Commands



10

In addition to the previously introduced categories of commands, there are commands that enable the user to manage database transactions:

- ☐ COMMIT—Saves database transactions
- ☐ ROLLBACK—Undoes database transactions
- ☐ SAVEPOINT—Creates points within groups of transactions in which to
- ☐ ROLLBACK
- ☐ SET TRANSACTION—Places a name on a transaction

Database Objects



11

A database object is any **defined object in a database that is used to store or reference data**. Some **examples of database objects include tables, views, clusters, sequences, indexes, and synonyms**. The table is the focus because it is the primary and simplest form of data storage in a relational database.

What Is a Schema? (logical representation of database)

A schema is a collection of database objects normally associated with one particular database username. This username is called the schema owner, or the owner of the related group of objects. You may have one or multiple schemas in a database. The user is only associated with the schema of the same name, and often the terms are used interchangeably. Basically, any user who creates an object has just created it in her own schema unless she/he specifically instructs it to be created in another one. So, based on a user's privileges within the database, the user has control over objects that are created, manipulated, and deleted. A schema can consist of a single table and has no limits to the number of objects that it may contain, unless restricted by a specific database implementation.

Schema cont...



12

Say you have been issued a database username and password by the database administrator. Your username is USER1. Suppose you log on to the database and then create a table called EMPLOYEE. According to the database, your table's actual name is USER1.EMPLOYEE. The schema name for that table is USER1, which is also the owner of that table. You have just created the first table of a schema. The good thing about schemas is that when you access a table that you own (in your own schema), you do not have to refer to the schema name. For instance, you could refer to your table as either one of the following:

- ☐ EMPLOYEE
- ☐ USER1.EMPLOYEE

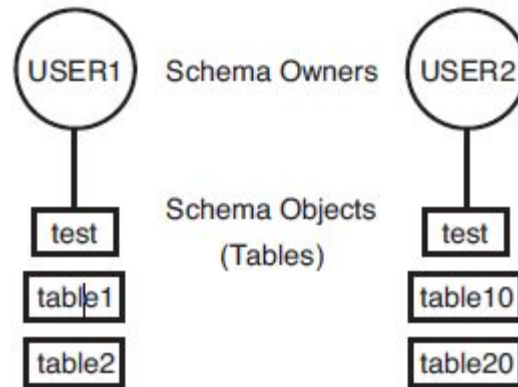
The first option is preferred because it requires fewer keystrokes. If another user were to query one of your tables, the user would have to specify the schema as follows:

USER1.EMPLOYEE

Schema cont...



13



In this example, both users have a table called **TEST**. Tables can have the same names in a database as long as they belong to different schemas. If you look at it this way, table names are always unique in a database because the schema owner is actually part of the table name. For instance, USER1.TEST is a different table than USER2.TEST. If you do not specify a schema with the table name when accessing tables in a database, the database server looks for a table that you own by default. That is, if USER1 tries to access TEST, the database server looks for a USER1-owned table named TEST before it looks for other objects owned by USER1, such as synonyms to tables in another schema.

Tables



14

The **table** is the primary storage object for data in a relational database. In its simplest form, a table consists of row(s) and column(s), both of which hold the data. A table takes up physical space in a database and can be permanent or temporary.

Note:

- ☐ When you create a table, you must specify the table name, name of each column, data type of each column, and size of each column
- ☐ The table and column names can be up to 30 characters long
- ☐ Table or column name must begin with a letter
- ☐ The names are not case sensitive
- ☐ Spaces and hyphens are not allowed in a table or a column name; but \$, _ and # are allowed

Columns



15

A **field**, also called a **column** in a relational database, is part of a table that is assigned a specific data type. The data type determines what kind of data the column is allowed to hold. This enables the designer of the table to help maintain the integrity of the data. Every database table must consist of at least one column. Columns are those elements within a table that hold specific types of data, such as a person's name or address. For example, a valid column in a customer table might be the customer's name. Generally, a column name must be one continuous string and can be limited to the number of characters used according to each implementation of SQL. It is typical to use underscores with names to provide separation between characters. For example, a column for the customer's name can be named `CUSTOMER_NAME` instead of `CUSTOMERNAME`. This is normally done to increase the readability of database objects. Columns also can be specified as **NULL** or **NOT NULL**, meaning that if a column is **NOT NULL**, something must be entered. If a column is specified as **NULL**, nothing has to be entered.

Rows



16

A row is a record of data in a database table. For example, a row of data in a customer table might consist of a particular customer's identification number, name, address, phone number, and fax number. A row is composed of fields that contain data from one record in a table. A table can contain as little as one row of data and up to as many as millions of rows of data or records.

Data Types



17

Data types are characteristics of the data itself, whose attributes are placed on fields within a table. For example, you can specify that a field must contain numeric values, disallowing the entering of alphanumeric strings. After all, you would not want to enter alphabetic characters in a field for a dollar amount. Defining each field in the database with a data type eliminates much of the incorrect data found in a database due to data entry errors. Field definition (data type definition) is a form of data validation that controls the type of data that may be entered into each given field.

Depending on the implementation of relational database management system (RDBMS), certain data types can be converted automatically to other data types depending upon their format. This type of conversion is known as an implicit conversion, which means that the database handles the conversion for you. An example of this is taking a numeric value of 1000.92 from a numeric field and inputting it into a string field. Other data types cannot be converted implicitly by the host RDBMS and therefore must undergo an explicit conversion. This usually involves the use of an SQL function, such as CAST or CONVERT. For example `SELECT CAST('12/27/1974' AS DATETIME) AS MYDATE`

Data Types cont...



18

The very basic data types, as with most other languages, are

- ❑ String types
- ❑ Numeric types
- ❑ Date and time types

String Types

- ❑ **char(n)** – The **fixed-length character** with user specified length n. The full form character can be used instead.
- ❑ **varchar(n)/varchar2(n)** - The **variable-length character** with user specified maximum length n. The full form character varying is equivalent.

Numeric Types

Numeric values are stored in fields that are defined as some type of number, typically referred to as NUMBER, INTEGER, REAL, DECIMAL, and so on.

Data Types cont...



19

Numeric Types

The following are the standards for SQL numeric values:

- ☐ BIT(n)
- ☐ BIT VARYING(n)
- ☐ DECIMAL(p,s)
- ☐ INTEGER
- ☐ SMALLINT
- ☐ BIGINT
- ☐ FLOAT(p, s)
- ☐ DOUBLE PRECISION(p, s)
- ☐ REAL(s)

Date and Time Types

Date and time data types are quite obviously used to keep track of information concerning dates and time. Standard SQL supports what are called DATETIME data types, which include the following specific data types:

- ☐ DATE
- ☐ TIME
- ☐ DATETIME
- ☐ TIMESTAMP

Other Data Types



20

NULL Types

NULL value is a missing value or a column in a row of data that has not been assigned a value. NULL values are used in nearly all parts of SQL, including the creation of tables, search conditions for queries, and even in literal strings.

LOB Types

Some variable-length data types need to hold longer lengths of data than what is traditionally reserved for a VARCHAR field. The **BLOB** and **TEXT** data types are two examples of such data types in modern database implementations. These data types are specifically made to hold large sets of data. The BLOB is a binary large object, so its data is treated as a large binary string (a byte string). A **BLOB** is especially useful in an implementation that needs to store binary media files in the database, such as images or MP3s. The **TEXT** data type is a large character string data type that can be treated as a large VARCHAR field. It is often used when an implementation needs to store large sets of character data in the database. An example of this would be storing HTML input from the entries of a blog site. Storing this type of data in the database enables the site to be dynamically updated.

DDL - Table Creation



21

The CREATE TABLE statement in SQL is used to create a table. Although the very act of creating a table is quite simple, much time and effort should be put into planning table structures before the actual execution of the CREATE TABLE statement. Carefully planning your table structure before implementation saves you from having to reconfigure things after they are in production. The basic syntax is:

```
CREATE TABLE table_name  
( field1 data_type [ not null ],  
  field2 data_type [ not null ],  
  field3 data_type [ not null ],  
  field4 data_type [ not null ],  
  field5 data_type [ not null ] );
```

[not null] depicts optional

DDL - Create Table Example



22

```
CREATE TABLE EMPLOYEE  
(EMP_ID CHAR(9) NOT NULL,  
EMP_NAME VARCHAR (40) NOT NULL,  
EMP_ST_ADDR VARCHAR (20) NOT NULL,  
EMP_CITY VARCHAR (15) NOT NULL,  
EMP_ST CHAR(2) NOT NULL,  
EMP_ZIP INTEGER,  
EMP_PHONE INTEGER NULL,  
EMP_PAGER INTEGER NULL);
```



Class Work – Create Tables

- | | | |
|----------------------------------|-----------------------------------|---------------------------------|
| ☐ STUDENT | ☐ CUSTOMER | ☐ PERSON |
| ☐ PRODUCT | ☐ ACCOUNT | |

DDL - Table Description



23

DESCRIBE statement is used for viewing table structure. The syntax is:

DESCRIBE table_name; or

DESC table_name;

For example: DESCRIBE EMPLOYEE;



Lab Exercise – Describe the following tables

- | | | |
|----------------------------------|-----------------------------------|---------------------------------|
| ☐ STUDENT | ☐ CUSTOMER | ☐ PERSON |
| ☐ PRODUCT | ☐ ACCOUNT | |

DDL - Dropping a Table



24

The syntax to drop a table follows:

`DROP TABLE table_name;`



Class Work – Drop the following tables

- | | | |
|----------------------------------|-----------------------------------|---------------------------------|
| ☐ STUDENT | ☐ CUSTOMER | ☐ PERSON |
| ☐ PRODUCT | ☐ ACCOUNT | |

DML - Inserting data into a Table



25

INSERT statement is to insert new data into a table. The basic syntax to begin:

```
INSERT INTO TABLE_NAME VALUES ('value1', 'value2', [ NULL ] );
```

Example: Let the table structure for “PRODUCT” is as follows –

COLUMN Name	Null?	DATA TYPE
PROD_ID	NOT NULL	VARCHAR(10)
PROD_DESC	NOT NULL	VARCHAR(25)
COST	NOT NULL	NUMBER(6,2)

```
INSERT INTO PRODUCT VALUES ('7725', 'LEATHER GLOVES', 24.99);
```



Class Work - Insert 2 distinct rows to the following tables

- ☐ STUDENT
- ☐ PRODUCT
- ☐ CUSTOMER
- ☐ ACCOUNT
- ☐ PERSON

DQL - Selecting data from a Table



26

The SELECT statement is used in conjunction with the FROM clause to extract data from the database in an organized, readable format. The SELECT part of the query is for selecting the data you want to see according to the columns in which they are stored in a table.

The syntax for a simple SELECT statement is as follows:

*Either * (means all columns) or Individual columns*

[TABLE2] depicts optional

SELECT [* | COLUMN1, COLUMN2] FROM TABLE1 [, TABLE2];

The SELECT keyword in a query is followed by a list of columns that you want displayed as part of the query output. The asterisk (*) denotes that all columns in a table should be displayed as part of the output.

Example - SELECT * FROM PRODUCT;

Class Work - select all rows from the following tables

- | | | |
|----------------------------------|-----------------------------------|---------------------------------|
| ☐ STUDENT | ☐ CUSTOMER | ☐ PERSON |
| ☐ PRODUCT | ☐ ACCOUNT | |

Experiments



27

Write PL/SQL queries for the following -

1. Create tables for - Employee, Student, Product, Customer, Account and Person. Identify relevant attributes for each table and link to respective tables. Ensure each table has a _ID column e.g. Employee should have EMPLOYEE_ID column, Student should have STUDENT_ID column etc.
2. Describe each table.
3. Insert at least 5 distinct rows to each table.
4. Fetch all data from the respective tables.
5. Drop each table





Thank You

End of Lab 1